

SMART-PLANT: SISTEMA IOT DE MONITOREO Y RIEGO AUTOMÁTICO DE PLANTAS EN ENTORNOS DOMÉSTICOS

Axel Valenzuela¹, Gabriela Bernard²

Resumen.

Smart-Plant es un sistema IoT de monitoreo ambiental y riego automático de plantas orientado a entornos domésticos y de oficina. El sistema utiliza módulos ESP32 para adquirir datos de humedad y temperatura del suelo y de nutrientes nitrógeno, fósforo y potasio (sensor NPK), transmitiendo la telemetría mediante el protocolo MQTT a un broker EMQX sobre una conexión TLS/SSL. Un servidor backend desarrollado en Node.js y Express procesa los mensajes MQTT, evalúa el estado de salud de cada planta contrastando las lecturas con una base de datos científicamente justificada de umbrales ideales para veinticinco especies agrupadas en cinco categorías ecológicas, y almacena la información en MongoDB mediante Mongoose. La arquitectura es modular y multinodo: cada dispositivo se identifica mediante su dirección MAC, permitiendo que un único servidor gestione simultáneamente múltiples módulos. Un dashboard web, alimentado por una API REST y un canal WebSocket, permite a usuarios autenticados visualizar el historial de lecturas y eventos de riego, así como activar el riego de forma manual. El sistema fue implementado sobre un circuito que integra un regulador de voltaje LM338, un módulo RS485/TTL y un relé de estado sólido para el control seguro de una bomba peristáltica. Los resultados confirman la viabilidad técnica de la comunicación end-to-end entre hardware, broker, backend y frontend, convirtiéndolo en un sistema modular funcional.

Palabras clave: Internet de las Cosas (IoT); monitoreo de plantas; riego automatizado; ESP32; MQTT; NPK; sistema multinodo.

1. Introducción

El avance del Internet de las Cosas (IoT) ha transformado la forma en que los sistemas embebidos interactúan con el entorno físico, permitiendo la recolección automatizada de datos y la ejecución de acciones en respuesta a condiciones detectadas por sensores. En el campo de la agricultura de precisión y el cuidado de plantas, estas tecnologías ofrecen una oportunidad para optimizar el uso del agua y mejorar las condiciones de crecimiento mediante sistemas inteligentes capaces de operar con mínima intervención humana (Angulo, 2025). Sin embargo, la adopción de estas tecnologías en contextos domésticos y de oficina —espacios con restricciones distintas a las de un invernadero o un cultivo extensivo— continúa siendo deficiente (Acosta y Cuaical, 2021), y las soluciones existentes tienden a limitarse a un único nodo, un único sensor o un control de riego

^{1,2} Estudiante de Ingeniería Electrónica y de Comunicaciones en el Instituto Tecnológico de Santo Domingo (INTEC).

basado exclusivamente en umbrales de humedad, sin ofrecer una interfaz que permita al usuario comprender el estado integral de su planta ni gestionar múltiples dispositivos de forma simultánea.

En este contexto, el presente trabajo describe el desarrollo de Smart-Plant, un sistema de monitoreo y riego automático de plantas basado en IoT, cuyo objetivo es medir humedad, temperatura y nutrientes NPK (nitrógeno, fósforo y potasio) de suelo y detectar en tiempo real la condición de humedad del suelo para activar el riego de manera autónoma cuando sea necesario. El sistema integra hardware de bajo costo y consumo (módulos ESP32) con tecnologías de comunicación estándar en IoT (protocolo MQTT) y una arquitectura de software escalable (Node.js, MongoDB), accesible a través de un dashboard web.

Uno de los aportes centrales del proyecto es su arquitectura modular y multimodo, lo que responde directamente a la ausencia de soluciones domésticas escalables capaces de gestionar múltiples nodos de monitoreo bajo una única interfaz interactiva con visualización histórica.

Además, a diferencia de los sistemas de riego convencionales orientados únicamente a la activación de una bomba, Smart-Plant incorpora un enfoque de diagnóstico ambiental: la telemetría de temperatura y NPK no determina la actuación del riego, pero permite generar alertas preventivas sobre el estado nutricional y térmico de la planta, aportando al usuario información que va más allá de la simple decisión de regar o no regar.

De esta manera, Smart-Plant no solo busca automatizar el riego de plantas en entornos domésticos, sino ofrecer una plataforma integral de monitoreo ambiental fundamentada científicamente, modular en su arquitectura de hardware y comunicaciones, y accesible a través de una interfaz web que permita al usuario tomar decisiones informadas sobre el cuidado de sus plantas sin requerir conocimientos técnicos previos sobre sensores o redes IoT.

1.1 Objetivos

Como objetivo general, se propone: diseñar e implementar un sistema embebido IoT modular que integre sensores de temperatura, humedad y nutrientes para el monitoreo ambiental y riego automatizado de plantas en espacios domésticos y de oficina, gestionado a través de internet mediante una interfaz de usuario interactiva.

Se definen, además, una serie de objetivos específicos que buscan ser cumplidos durante el desarrollo del proyecto:

1. Acondicionar y calibrar de manera adecuada sensores de humedad, temperatura y nutrientes para la adquisición de datos precisos del suelo en tiempo real.
2. Diseñar y crear una base de datos para almacenar los umbrales ideales de las variables ambientales de distintas plantas, en base a los cuales se manejará el riego automático y se definirán alertas de riesgo de las plantas.
3. Establecer una infraestructura de comunicación vía WiFi basada en el protocolo MQTT para transmitir la telemetría a un servidor y recibir comandos de actuación.

4. Crear una aplicación/dashboard que permita visualizar el histórico de riegos y mediciones, el estado actual de las plantas, y que cuente con un control para activar el riego manualmente.
5. Configurar el firmware y el servidor para que el sistema sea modular, permitiendo el emparejamiento y la gestión de múltiples nodos de monitoreo en la misma red.

1.2 Justificación

El cuidado de plantas en entornos domésticos y de oficina frecuentemente se ve comprometido por la falta de tiempo, el desconocimiento de las necesidades hídricas y ambientales específicas de cada planta o la ausencia prolongada de los usuarios. Esta situación no solo repercute negativamente en la salud y supervivencia de las plantas, sino que también promueve prácticas ineficientes como el riego excesivo, lo cual genera un desperdicio innecesario de agua. Aunque la agricultura de precisión ha demostrado cómo la tecnología puede optimizar el uso de recursos hídricos y mejorar las condiciones de crecimiento mediante sistemas autónomos, la adopción de estos mecanismos en el ámbito doméstico aún es muy limitada (Acosta y Cuaical, 2021).

La revisión del estado del arte evidencia que la mayoría de los sistemas IoT actuales están fuertemente orientados a cultivos agrícolas a gran escala o invernaderos, tal y como se evidencia en Tovar Soto et al. (2019). Existe una carencia de soluciones domésticas que sean modulares, escalables y que ofrezcan una interfaz de usuario interactiva capaz de gestionar múltiples nodos simultáneamente y visualizar datos históricos.

Además, a diferencia de los sistemas convencionales orientados exclusivamente al control de riego, este proyecto incorpora un sensor multiparámetro capaz de adquirir datos en tiempo real de humedad, temperatura, y nutrientes (nitrógeno, fósforo y potasio - NPK). La combinación de estas mediciones permite realizar un monitoreo más preciso, facilitando la detección de necesidades de riego y la evaluación de las condiciones adecuadas para el desarrollo de la planta. Aunque la activación autónoma de la bomba de agua depende únicamente de los umbrales de humedad del suelo, la captura y procesamiento de las variables de, temperatura y NPK se justifican desde un enfoque preventivo y de diagnóstico.

En este contexto, el desarrollo de Smart-Plant se justifica por la necesidad de facilitar el acceso a tecnologías de monitoreo ambiental y actuación automatizada en espacios interiores que faciliten el mantenimiento hídrico de las plantas y permitan al usuario tomar decisiones informadas sobre el estado de su planta. Al integrar plataformas de hardware de bajo costo y consumo, como el microcontrolador ESP32, con arquitecturas de comunicación estandarizadas basadas en el protocolo MQTT, el proyecto ofrece una alternativa accesible, modular y funcional para la mayoría de los usuarios.

2. Marco Teórico

2.1 La nutrición mineral en plantas: fundamentos y relevancia

El crecimiento y desarrollo de las plantas depende de la disponibilidad y absorción de dieciséis elementos minerales esenciales. De estos, el nitrógeno (N), el fósforo (P) y el potasio (K) se clasifican como macronutrientes primarios, ya que son requeridos en concentraciones superiores a cualquier otro elemento inorgánico y constituyen los factores más frecuentes de limitación del crecimiento en condiciones naturales y de cultivo (*Marschner, 2012; Epstein & Bloom, 2005*). La deficiencia de cualquiera de estos tres nutrientes produce síntomas visuales característicos y reduce la producción vegetal de manera significativa; inversamente, concentraciones excesivas pueden generar toxicidad directa o interferir con la absorción de otros elementos mediante competencia iónica (*Marschner, 2012*).

El conocimiento preciso de los rangos de concentración óptimos de N, P y K en el suelo o sustrato constituye la base sobre la cual se diseñan los programas de fertilización y, en el contexto de la agricultura de precisión, los sistemas automatizados de apoyo al cultivo. La medición continua de estas variables mediante sensores electrónicos integrados en arquitecturas IoT hace posible implementar estrategias de respuesta en tiempo real al estado nutricional de la planta, superando las limitaciones de los análisis de laboratorio de periodicidad semanal o mensual que no capturan la variabilidad dinámica de los nutrientes en el suelo (*Gondchawar & Kawitkar, 2016*).

2.2 Variables ambientales: temperatura y humedad relativa

La temperatura del suelo condiciona directamente la absorción radicular de N, P y K, la actividad microbiana responsable de la mineralización de la materia orgánica y la velocidad de las reacciones enzimáticas en la rizosfera (*Marschner, 2012*). Por debajo de 10°C la absorción iónica se inhibe significativamente aun cuando los nutrientes estén disponibles, mientras que temperaturas superiores a 35°C generan estrés térmico radicular y aceleran la lixiviación de compuestos solubles (*Taiz et al., 2015*). El rango óptimo para la mayoría de los cultivos hortícolas se sitúa entre 18 y 25°C, con variaciones según la categoría vegetal: los cultivos de clima frío como la lechuga prefieren suelos entre 10 y 18°C, mientras que las plantas de origen tropical admiten valores de hasta 30°C (*Epstein & Bloom, 2005*).

La humedad del suelo determina la disponibilidad y movilidad de los iones nutrientes en la solución del suelo. El N, P y K son absorbibles por las raíces únicamente cuando se encuentran disueltos en agua intersticial; un suelo excesivamente seco impide su absorción a pesar de que las concentraciones totales sean suficientes. Inversamente, un suelo saturado desplaza el aire del espacio poroso, generando condiciones anaeróbicas que inhiben la respiración radicular, promueven la lixiviación de nitratos y favorecen patógenos de raíz como *Phytophthora* spp. (*Marschner, 2012*). En el sistema Smart-Plant, la humedad del suelo es medida mediante un sensor capacitivo enterrado, cuyo principio de operación consiste en cuantificar la constante dieléctrica del sustrato, la cual varía de

forma predecible con el contenido volumétrico de agua (VWC). Este método es no invasivo, evita la electrólisis de los electrodos que afecta a los sensores resistivos y ofrece lecturas estables en el tiempo, representándose la humedad como porcentaje de VWC en las alertas del sistema.

2.3 Caracterización de umbrales nutricionales y ambientales por categoría vegetal

La heterogeneidad de requerimientos entre tipos de plantas obliga a definir umbrales específicos por especie o grupo ecológico, en lugar de adoptar valores universales. La literatura agronómica sistematiza esta información mediante tablas de interpretación de análisis de suelo, que clasifican los niveles de N, P y K como deficientes, bajos, óptimos o excesivos según el cultivo, basándose en experimentos de respuesta a la fertilización con replicación estadística en múltiples sitios y temporadas (*OSU Extension, s.f.; UC ANR, 2016*). Esta metodología permite distinguir cinco perfiles de requerimientos diferenciados, que sirven como marco de referencia para el diseño de la base de datos de umbrales del sistema Smart-Plant.

Las plantas de follaje tropical un perfil de demanda moderada y equilibrada, con énfasis en N para sostener el crecimiento foliar continuo, temperaturas óptimas de 15 a 32°C y HR preferida superior al 50%, aunque *Sansevieria* tolera HR de apenas 30-50% por su origen semiárido (*Sonneveld & Voogt, 2009*). Las plantas florales exhiben mayor variabilidad interna: la rosa presenta alta demanda de N y K para la producción de flores de calidad comercial (*Sonneveld & Voogt, 2009*), mientras que las orquídeas del género *Phalaenopsis* son extraordinariamente sensibles a la acumulación de sales en el sustrato de corteza de pino, operando con concentraciones de N de apenas 25 a 100 mg/kg (*AOS, 2023*). Las plantas desérticas ocupan el extremo de menor demanda nutricional en todas las variables, y el riesgo de daño por fertilización excesiva es más frecuente que el de deficiencia (*Marschner, 2012*). Los cultivos hortícolas presentan los umbrales de N, P y K más elevados del conjunto, reflejo de sus altas tasas de crecimiento y producción de biomasa cosechable, con rangos de temperatura y HR óptimos más estrechos que los de las ornamentales. Finalmente, los cultivos frutales comparten con las hortalizas la alta demanda de K pero exhiben requerimientos de P generalmente menores, y sus rangos óptimos de temperatura varían significativamente según el origen geográfico: los frutales tropicales y subtropicales como el limón y la mandarina requieren 20-30°C, mientras que el durazno y el arándano se desarrollan mejor entre 15 y 24°C (*Strik, 2016*).

Tabla 1.

Base de datos de plantas del sistema con rangos óptimos de variables ambientales.

Planta	Categoría	Temp (°C)	HR (%)	N (mg/kg)	P (mg/kg)	K (mg/kg)
Monstera	Follaje	18–27	50–70	75–150	20–30	75–150
Pothos	Follaje	15–29	50–70	50–100	15–25	50–100
Begonia rex	Follaje	18–24	50–70	75–125	20–30	75–125

Helecho	Follaje	16–24	60–80	50–100	15–25	50–100
Sansevieria	Follaje	18–32	30–50	50–100	15–20	50–100
Orquídea	Florales	18–29	50–70	25–100	25–87	50–168
Rosa	Florales	15–26	60–70	100–150	30–75	150–200
Tulipán	Florales	5–18	50–70	50–100	30–60	100–150
Geranio	Florales	15–25	40–60	75–125	25–75	75–125
Hortensia	Florales	15–21	60–80	75–100	25–50	75–150
Cactus	Desérticas	21–35	10–30	5–25	5–15	15–40
Cactus barril	Desérticas	21–38	10–25	5–20	5–10	10–30
Aloe vera	Desérticas	13–27	20–40	5–25	5–15	15–40
Euphorbia milii	Desérticas	18–24	30–50	10–25	10–20	10–25
Planta jade	Desérticas	18–24	30–50	5–25	5–15	15–40
Tomate	Vegetales	18–27	50–70	20–40	30–65	150–250
Batata	Vegetales	24–30	50–70	20–35	30–65	150–200
Cebolla	Vegetales	13–24	50–70	15–30	30–65	150–200
Zanahoria	Vegetales	15–21	50–70	15–25	30–65	150–200
Lechuga	Vegetales	15–22	50–70	20–35	30–65	150–250
Durazno	Frutales	15–28	50–70	20–40	10–30	100–200
Arándano	Frutales	15–24	60–70	20–40	10–25	80–150
Limón	Frutales	20–30	50–70	20–40	10–20	80–160
Fresa	Frutales	15–22	60–80	30–80	40–93	150–254
Mandarina	Frutales	20–30	50–70	20–40	10–20	80–160

Nota. Elaboración propia.

2.4 Antecedentes en sistemas IoT de monitoreo y riego automático

El monitoreo y riego automatizado mediante IoT ha sido abordado desde múltiples perspectivas en la literatura reciente, con énfasis en tres dimensiones principales: la plataforma de hardware empleada, el protocolo de comunicación adoptado y la estrategia de control utilizada. A continuación, se resumen los trabajos encontrados considerados más relevantes para el proyecto.

2.4.1 Plataformas de hardware predominantes

En el ámbito de IoT doméstico y agrícola, los microcontroladores más citados en trabajos recientes son los basados en la familia ESP-32 y, en menor medida, variantes de

Arduino con módulo Wi-Fi (p. ej., ESP8266 como núcleo de red). El ESP32-WROOM-32U se ha consolidado como plataforma de referencia en sistemas de riego con monitoreo IoT, al integrar procesamiento, Wi-Fi 802.11 y Bluetooth en un solo módulo, con memoria RAM suficiente y múltiples interfaces (SPI, I2C, UART) para la conexión de sensores y actuadores.

2.4.2 Protocolos de comunicación y herramientas de software

Para el transporte de datos desde los nodos sensores hasta un servidor o broker local, el protocolo MQTT (con implementaciones como Mosquitto o HiveMQ) aparece de manera consistente en la literatura como la opción más adecuada para IoT doméstico, dado su diseño ligero, orientado a eventos y con bajo overhead. Su modelo de publicación/suscripción lo hace especialmente apto para la transmisión de telemetría de sensores de humedad, temperatura y estado de actuadores.

2.4.3 Sistemas de riego automatizado con umbrales fijos

Diversos autores han implementado sistemas de riego automatizado para huertos caseros usando ESP32 como nodo sensor y Raspberry Pi como broker MQTT y servidor de reglas (Hernández González et al., 2025; Acosta y Cuaical, 2021; Perea Alvarado y Bustamante Briones, 2024). Otros trabajos extienden este enfoque a invernaderos y cultivos protegidos, incorporando sensores de humedad del suelo (FC-28) y decisiones autónomas basadas en umbrales de humedad y climatología (Angulo, 2025; Gómez Zea et al., 2025). En el ámbito del código abierto, repositorios como RiegoJardin-ESP32 y Smart Farming ESP32 —aunque sin revisión académica formal— constituyen referencias técnicas relevantes que implementan lecturas de sensores DHT11 y FC-28, accionamiento de relés y visualización mediante NodeRED (Richard-Roman, 2026; lily-osp, 2025). Estos antecedentes demuestran la viabilidad técnica del enfoque y han permitido cuantificar sus beneficios: Hasib y Ahsanul Sarkar Akib (2026), en una evaluación con plantas de interior y macetas, lograron una precisión del 92 % en la humedad óptima del suelo y una reducción del 40 % en el consumo de agua respecto al riego manual tradicional.

2.4.4 Sistemas predictivos e inteligencia artificial

Más allá del control por umbrales estáticos, trabajos recientes proponen estrategias predictivas que incorporan aprendizaje automático. Ruiz (2025) desarrolla un sistema híbrido en dos capas: un Arduino Nano recolecta datos de humedad capacitiva, temperatura y humedad ambiental y los envía inalámbricamente a una Raspberry Pi 4 local, donde un modelo LSTM cruza la telemetría de las macetas con el pronóstico climático de Open-Meteo para anticipar el riego antes de que la planta alcance estrés hídrico. En una línea complementaria, Oppong (2025) propone migrar los modelos de decisión directamente al microcontrolador, eliminando la dependencia de la nube y reduciendo la latencia y la vulnerabilidad ante cortes de conectividad.

2.4.5 Brechas identificadas

A pesar de los avances documentados, la revisión identifica que la mayoría de los sistemas están orientados a cultivos agrícolas o invernaderos de mediana y gran escala. Se identifica una ausencia de implementaciones en entornos domésticos que combinen escalabilidad modular para la gestión simultánea de múltiples nodos de monitoreo con una interfaz de usuario interactiva que permita visualización histórica y control remoto. Estas brechas se corresponden con los objetivos del presente proyecto: la arquitectura modular propuesta y el dashboard de control buscan aportar novedad y utilidad al sistema propuesto.

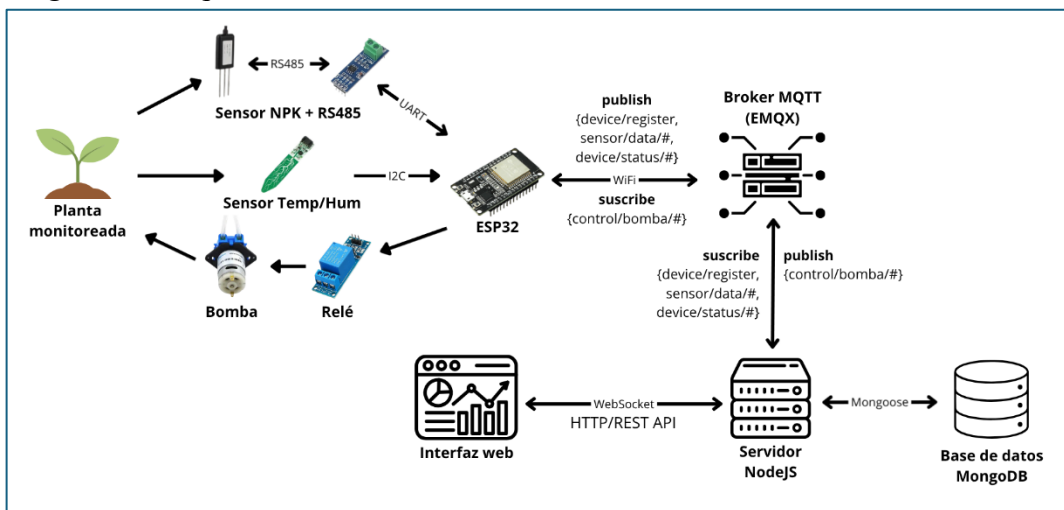
3. Arquitectura del sistema

La arquitectura del sistema Smart-Plant ha sido diseñada bajo un paradigma de IoT distribuido, separando las responsabilidades en capas físicas, de comunicaciones y de aplicación.

En la Figura 1 se ilustra el diagrama general de la arquitectura del sistema.

Figura 1.

Diagrama de arquitectura del sistema.



La arquitectura se divide en las siguientes capas:

1. **Capa de recepción (Hardware):** Compuesta por un módulo central basado en el microcontrolador ESP32, el cual interactúa directamente con el entorno a través de los sensores del suelo y actúa mediante una bomba de agua.
2. **Capa de red (Comunicaciones):** Utiliza WiFi para conectar al ESP32 a internet. La transferencia de datos de telemetría y recepción de comandos de riego se gestiona mediante el protocolo MQTT.
3. **Capa de aplicación y datos (Backend/Frontend):** Un servidor desarrollado en NodeJS organiza la lógica de intercambio, procesando los mensajes MQTT a través del broker EMQX, y almacenando los registros históricos y eventos de riego en una base de datos

no relacional MongoDB. Finalmente, un dashboard web permite al usuario visualizar el estado y enviar comandos.

4. Tecnologías seleccionadas

A continuación, se detallan cada una de las tecnologías, tanto de hardware como de software, seleccionadas y utilizadas para el desarrollo del proyecto. Cada una de estas tecnologías fue seleccionada en base a ciertos criterios, favoreciendo aquellas herramientas cuyos parámetros se ajustarán a los requerimientos de nuestro proyecto en términos de eficiencia y funcionalidad, así como las opciones más económicas para reducir el costo del proyecto.

4.1 Hardware seleccionado

Para el hardware se tomaron dos decisiones importantes: el microcontrolador elegido y el sensor de humedad, dado que de estos dos componentes depende en gran medida el sistema propuesto. A continuación, se detallan los componentes elegidos y su justificación.

4.1.1 Microcontrolador

Se seleccionó el ESP32 como microcontrolador principal debido a que integra conectividad Wi-Fi, característica fundamental para el proyecto, ya que permite enviar los datos de los sensores hacia una base de datos en tiempo real y recibir instrucciones desde la interfaz del sistema. Gracias a esta conectividad, el usuario puede monitorear el estado de la planta y acceder a la información desde la plataforma de manera remota.

Además, el ESP32 ofrece buena capacidad de procesamiento, múltiples pines de entrada y salida y bajo consumo energético, lo que permite conectar varios sensores y actuadores al mismo tiempo sin afectar el rendimiento. También resulta conveniente por su facilidad de programación y por la amplia disponibilidad de documentación y soporte para su implementación.

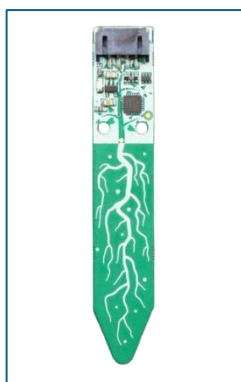
4.1.2 Sensor de temperatura/humedad

Se escogió el Adafruit STEMMA Soil Sensor – I2C Capacitive Moisture Sensor porque permite obtener, mediante un solo dispositivo, una medición capacitiva de la humedad del sustrato y una lectura aproximada de la temperatura ambiental. A diferencia de los sensores resistivos tradicionales, su sonda no posee metales expuestos ni aplica corriente continua al suelo, lo que reduce la oxidación, el deterioro y la necesidad de recalibraciones frecuentes; además, su interfaz I²C y su conector JST-PH facilitan la integración con microcontroladores y sistemas de monitoreo automatizado (Adafruit Industries, 2024). La humedad es una variable esencial porque permite determinar la cantidad de agua disponible en la zona de las raíces, identificar cuándo la planta necesita riego y evitar tanto el estrés hídrico como la saturación del suelo, condiciones que pueden

afectar la absorción de agua, oxígeno y nutrientes (Zamora Re et al., 2026). Por otro lado, la temperatura influye directamente en procesos fisiológicos como la fotosíntesis, la transpiración, la respiración, la germinación y la floración, por lo que su seguimiento permite detectar condiciones ambientales desfavorables y tomar decisiones oportunas para proteger el crecimiento de la planta (VanDerZanden, 2008/2024). No obstante, la lectura de temperatura de este sensor proviene del microcontrolador interno y tiene una precisión aproximada de ± 2 °C, por lo que resulta más apropiada para observar tendencias y cambios ambientales que para realizar mediciones científicas de alta precisión.

Figura 2.

Modelo utilizado del sensor temperatura/humedad



4.1.3 Sensor NPK (Nitrógeno, Fósforo y Potasio)

Se seleccionó el Soil NPK sensor, un sensor de alta precisión ambiental multiparámetro capaz de medir nitrógeno, fósforo y potasio (NPK). Estas variables son fundamentales para evaluar el estado de la planta y las condiciones del sustrato donde se desarrolla. Se escogió este sensor por facilidad de acceso, costo y su alta precisión puesto a que es destinado a cultivos de invernaderos profesionales, también tiene una salida 485 que facilita la integración con diversos sistemas de automatización industrial, mejorando la comunicación de datos y la compatibilidad.

El nitrógeno, fósforo y potasio son macronutrientes esenciales para el crecimiento vegetal, ya que participan en procesos como la síntesis de clorofila, el desarrollo radicular, la transferencia de energía y la regulación hídrica de la planta (Taiz et al., 2015).

La combinación de estas mediciones permite realizar un monitoreo más preciso, facilitando la detección de necesidades de nutrientes para el desarrollo de la planta.

Figura 3.

Modelo utilizado de sensor NPK



4.1.4 Módulo RS485/TTL

Para conectar el sensor NPK al microcontrolador se utiliza un módulo MAX485, ya que el sensor transmite los datos mediante el estándar industrial RS-485, mientras que microcontroladores como el ESP32 trabajan con señales seriales TTL/UART. El módulo convierte las señales diferenciales A y B del RS-485 en señales TX y RX compatibles con el microcontrolador, permitiendo una comunicación estable, resistente al ruido eléctrico y adecuada para cables de mayor longitud. Además, controla la dirección de la comunicación mediante los pines RE y DE, lo que permite alternar entre el envío de solicitudes al sensor y la recepción de las mediciones de nitrógeno, fósforo y potasio.

Figura 4.

Modelo utilizado del Módulo RS485/TTL



4.1.5 Actuadores (relé + bomba peristáltica)

Para el sistema de riego automatizado se seleccionó un relé junto con una bomba peristáltica. El relé permite que el ESP32 controle el encendido y apagado de la bomba de manera segura, ya que actúa como un interruptor entre el microcontrolador y el sistema de alimentación de la bomba, protegiendo el circuito y facilitando la automatización. El relé utilizado fue uno genérico de estado sólido SRD-05VDC-SL-C.

La bomba peristáltica NKP-DC-S108 de Kamoer fue elegida porque permite suministrar cantidades controladas de agua con precisión, lo que resulta ideal para el cuidado de plantas. Se alimenta de 12V y consumiría 5W máximo. El objetivo del sistema es que, cuando la planta necesite riego, el usuario tenga la opción de activar la bomba y que esta se apague automáticamente una vez que se haya suministrado la cantidad de agua necesaria, determinada según la información almacenada en la base de datos para cada tipo de planta.

Figura 5.

Bomba peristáltica Kamoer NKP-DC-S10B.



Figura 6.

Relé SRD-05VDC-SL-C



4.2 Stack de Software seleccionado

A continuación, se detallan las tecnologías de software utilizadas para el desarrollo del proyecto, las cuales se reducen a elegir el protocolo de comunicación para transmitir la telemetría, el lenguaje en que se programará el servidor y qué base de datos se utilizará para almacenar los datos del servidor.

4.2.1 Protocolo y broker MQTT

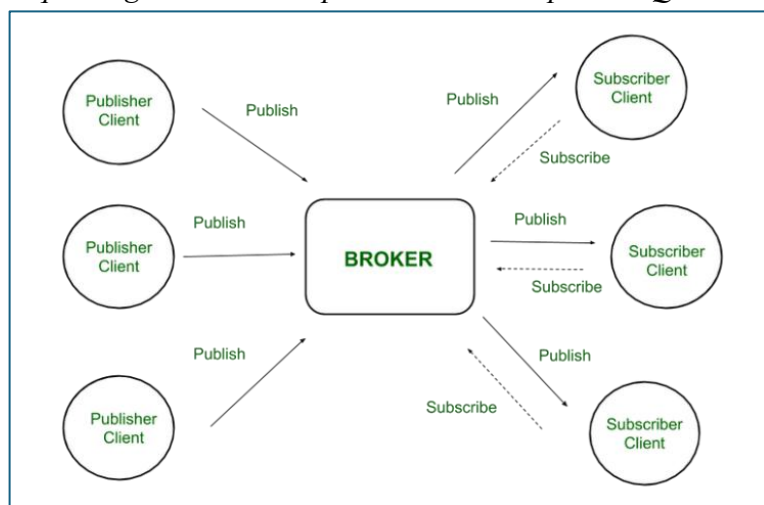
Para la comunicación entre los nodos ESP32 y el servidor, se adoptó el protocolo MQTT por su bajo overhead y su arquitectura de pub/sub, ideal para dispositivos con recursos limitados. Como broker MQTT se seleccionó EMQX, que ofrece alta disponibilidad, soporte nativo para clústeres, panel de administración integrado y soporte para múltiples protocolos (MQTT, WebSocket, HTTP). EMQX fue evaluado frente a Mosquitto y HiveMQ, siendo seleccionado por su interfaz de administración y capacidades de monitoreo en tiempo real.

MQTT es un protocolo de mensajería simple y liviano que se utiliza para establecer comunicación entre múltiples dispositivos. Es un protocolo basado en TCP que se basa en el modelo de publicación-suscripción. Este protocolo de comunicación es adecuado para transmitir datos entre dispositivos con recursos limitados que tienen un ancho de banda bajo y requisitos de energía bajos. Por lo tanto, este protocolo de mensajería se usa ampliamente

para la comunicación en IoT Framework (GeeksforGeeks, 2020).

Figura 7.

Esquema general modelo publicación-suscripción MQTT



Nota. Tomado de Introduction of Message Queue Telemetry Transport Protocol (MQTT), por GeeksforGeeks, 2025.

El modelo MQTT de publicación-suscripción es un patrón de mensajería que involucra clientes (dispositivos que establecen una conexión con el broker) que envían o reciben mensajes sobre topics (Prokosch, 2025). Los clientes no necesitan estar al tanto de que otros clientes envían o reciben mensajes. La comunicación MQTT involucra al menos a tres partes:

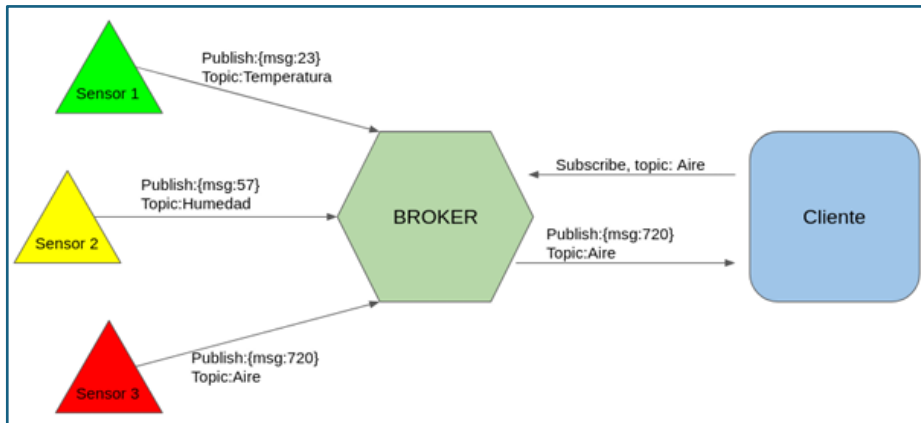
- **Publicadores:** Dispositivos o aplicaciones que envían mensajes que contienen datos sobre topics específicos sin el conocimiento de los destinatarios (suscriptores).
- **Suscriptores:** Dispositivos o aplicaciones que expresan interés en recibir datos publicados sobre topics específicos “suscribiéndose” a estos topics.
- **MQTT Broker:** La pieza central que recibe mensajes de los publicadores y los entrega a los suscriptores en función de sus respectivas suscripciones.

La Figura 8 muestra un diagrama que ilustra este proceso en funcionamiento de manera general.

El protocolo opera sobre TCP/IP con un overhead de cabecera mínimo (2 bytes fijos), lo que lo hace eficiente para dispositivos con recursos limitados. MQTT posee además un mecanismo para asegurar que los mensajes son recibidos, y es el QoS (calidad de servicio). Soporta tres niveles de calidad de servicio: at most once (QoS 0), at least once (QoS 1) y exactly once (QoS 2).

Figura 8.

Diagrama de ejemplo de funcionamiento MQTT



Nota. Tomado de Diseño y desarrollo de una aplicación web de monitorización de dispositivos Bluetooth Low Energy, por D. Ros López, 2021, Universidad Politécnica de Cartagena.

4.2.2 Servidor Backend

El servidor de aplicaciones fue implementado en Node.js, un entorno de ejecución de JavaScript del lado del servidor construido sobre el motor V8 de Google, con un modelo de entrada/salida no bloqueante basado en un bucle de eventos (event loop). Esta arquitectura le permite gestionar de forma eficiente múltiples conexiones concurrentes, entre las que se cuentan el cliente MQTT, los WebSocket abiertos con los navegadores y las operaciones de lectura/escritura en la base de datos, sin recurrir al modelo de hilos del sistema operativo, resultando en un bajo consumo de memoria por conexión.

Se emplearon los paquetes `mqtt.js` para la integración con el broker EMQX, `mongoose` para el modelado de datos en MongoDB, y `express.js` para la exposición de la API REST consumida por el frontend.

Para la comunicación bidireccional entre el servidor y el dashboard web se implementó un servidor WebSocket utilizando la biblioteca `ws`. El protocolo WebSocket establece un canal de comunicación full-duplex y persistente sobre una única conexión TCP, permitiendo al servidor enviar datos al navegador de forma inmediata (server push) sin que el cliente realice polling (GeeksforGeeks, 2019). Esto resulta esencial para un sistema IoT donde los datos de los sensores se generan continuamente y deben reflejarse en el dashboard con la mínima latencia posible.

Cuando llega un mensaje MQTT, el servidor lo retransmite en broadcast a todos los clientes WebSocket activos; en sentido inverso, los comandos enviados desde el dashboard viajan al servidor vía WebSocket y son enrutados al tópico MQTT del dispositivo correspondiente.

4.2.3 Base de datos

Como sistema de gestión de base de datos se seleccionó MongoDB en su modalidad de servicio en la nube MongoDB Atlas, una base de datos NoSQL orientada a documentos

que almacena registros en formato BSON (MongoDB, s.f.). El modelo de datos orientado a documentos se adapta de manera natural a las lecturas de sensores IoT, donde la estructura de cada lectura puede variar entre dispositivos o a lo largo del tiempo del proyecto (por ejemplo, al agregar nuevos tipos de sensores), sin requerir migraciones de esquema complejas.

Para el modelado de datos y la interacción con MongoDB desde Node.js se utilizó Mongoose, un ODM (Object Document Mapper) que permite definir esquemas tipados con validación automática, índices y métodos de consulta encadenables. La definición de esquemas explícitos en Mongoose, a pesar de la flexibilidad inherente de MongoDB, aporta consistencia de datos y documentación implícita de la estructura esperada para cada colección.

5. Descripción del proyecto

Esta sección se enfoca en describir el funcionamiento de las principales partes y funcionalidades del proyecto: la infraestructura de comunicaciones, el sistema multinodo para asignación de plantas, el sistema de autenticación, el servidor backend y el circuito implementado.

5.1 Infraestructura de red

Para la capa de red, el ESP32 se conecta a internet por medio de WiFi usando la librería `WiFiClientSecure` y establece una conexión segura (TLS/SSL a través del puerto 8883) con un broker MQTT alojado en la nube en la plataforma EMQX.

El firmware ejecuta la conexión en la función `setupNetwork()`, mediante la llamada a `WiFi.begin(SSID, WIFI_PASSWORD)`, seguida de un ciclo de espera que verifica el estado `WiFi.status() != WL_CONNECTED` con un intervalo de 500 ms entre verificaciones. Si no logra conectarse en el intervalo establecido, se reinicia el ESP32 para volver a intentar la conexión.

En el momento en que el ESP32 logra conectarse a la red, se extrae su MAC Address y se suscribe a los tópicos correspondientes del broker MQTT para iniciar la comunicación con el servidor.

Figura 9.

Función de configuración y establecimiento de la conexión.

```
void setupNetwork() {
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  unsigned long startAttemptTime = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - startAttemptTime <
15000) {
    delay(500);
  }
  if (WiFi.status() != WL_CONNECTED) {
    ESP.restart();
  }
}
```

```

deviceID = WiFi.macAddress();
topicData    = "sensor/data/" + deviceID;
topicControl = "control/led/" + deviceID;
mqtt.setServer(MQTT_BROKER_URL, MQTT_PORT);
mqtt.setCallback(mqttCallback);
}

```

Nota. Elaboración propia.

Para el manejo de desconexiones se implementó la función ***loopNetwork()***, la cual se muestra en la Figura 10. Esta función se ejecuta constantemente en el loop principal, verificando si hubo una desconexión para llamar a la función ***setupNetwork()*** y volver a establecer la conexión.

Figura 10.

Función de reconexión.

```

void loopNetwork() {
    if (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        ESP.restart();
    }
    if (!mqtt.connected()) connectMQTT();
    mqtt.loop();
}

```

Nota. Elaboración propia.

Se estableció la comunicación MQTT entre el módulo ESP32 y el broker EMQX Cloud utilizando una conexión cifrada TLS/SSL sobre el puerto 8883. En el firmware, el cliente MQTT se construye sobre una instancia *WiFiClientSecure* y la biblioteca *PubSubClient*, que implementa el protocolo MQTT.

La gestión del ciclo de vida de la conexión MQTT se implementa en la función ***connectMQTT()***, como se observa en la Figura 10, que ejecuta un bucle de reconexión mientras el cliente no esté conectado. Al establecer la sesión exitosamente, el cliente se suscribe a sus tópicos de control y publica el mensaje de registro del dispositivo:

Figura 11.

Función de conexión MQTT.

```

void connectMQTT() {
    unsigned long startAttemptTime = millis();
    String topicStatus = "device/status/" + deviceID;
    while (!mqtt.connected() && millis() - startAttemptTime < 10000) {
        if (mqtt.connect(deviceID.c_str(), MQTT_USERNAME, MQTT_PASSWORD,
            topicStatus.c_str(), 0, true, "offline")) {
            mqtt.subscribe(topicControl.c_str());
            mqtt.publish(topicStatus.c_str(), "online", true);
            String reg = "{\"device_id\":\"" + deviceID +
                "\", \"status\":\"pending\"}";
            mqtt.publish(topicRegister.c_str(), reg.c_str(), false);
        }
    }
}

```



```

    } else {
        delay(2000);
    }
}
if (!mqtt.connected()) {
    ESP.restart();
}
}
}

```

Nota. Elaboración propia.

5.2 Lectura y envío de telemetría

El archivo `hardware.cpp` encapsula la lógica para interactuar con los sensores físicos y extraer las métricas de humedad, temperatura y macronutrientes del suelo. Para mitigar el ruido y mejorar la precisión de los datos, el firmware aplica un filtro promediador.

4.2.1 Adquisición de humedad y temperatura.

El módulo ESP32 se comunica vía I2C (usando la biblioteca `Adafruit_seesaw`) con el sensor de humedad capacitivo, el cual también integra un termómetro. La medición de humedad capacitiva es sensible a las fluctuaciones de temperatura. Por ello, el firmware captura primero el valor crudo de capacitancia y luego lee la temperatura actual. Utilizando una constante de compensación térmica ($\alpha = -0.0046$), el sistema corrige la capacitancia leída respecto a una temperatura de referencia de 25 °C.

La capacitancia corregida se mapea luego a un porcentaje de humedad (0% a 100%) utilizando valores de calibración empíricos definidos en el código. Esta calibración se basa en registrar el valor de capacitancia del sensor cuando está expuesto a tierra completamente seca (***SUELO_SECO*** = 400) y su valor cuando está sumergido en tierra completamente húmeda (***SUELO_HUMEDO*** = 990).

Figura 12.

Bloque de código para lectura del sensor de humedad.

```

float leerCapacitanciaPromedio() {
    float suma = 0;
    for (int i = 0; i < 10; i++) {
        suma += ss.touchRead(0);
        delay(10);
    }
    return suma / 10.0;
}

float corregirPorTemperatura(float capRaw, float tempC) {
    float factor = 1.0 + ALPHA * (tempC - T_REF);
    if (factor <= 0.0) factor = 0.01;
    return capRaw / factor;
}

void processHumidity_Temp(int &humedadOut, int &tempOut) {
    float tempC = ss.getTemp();

```

```

float capRaw = leerCapacitanciaPromedio();
float capCorregida = corregirPorTemperatura(capRaw, tempC);
int humidity = calcularHumedad(capCorregida);
humedadOut = humidity;
tempOut = (int)tempC;
}

```

Nota. Elaboración propia.

4.2.2 Lectura de nutrientes (Sensor NPK).

El sensor NPK opera mediante el protocolo RS485 Modbus RTU, el cual requiere que el ESP32 transmita tramas de bytes específicas para solicitar el nivel de cada nutriente. El sistema alterna dinámicamente los pines de transmisión y recepción (DE y RE) para controlar el flujo de datos. Al ejecutar la función *readNPKValues()*, el microcontrolador envía consecutivamente las tramas predefinidas para solicitar nitrógeno, fósforo y potasio, introduciendo un retardo entre cada petición para permitir que el sensor procese la respuesta. El sistema lee la trama de respuesta (7 bytes), valida la cabecera Modbus y extrae el valor del nutriente combinando el byte alto y bajo recibidos.

Figura 13.

Función para leer el sensor de nutrientes NPK..

```

uint16_t leerNPK(const byte* comando, size_t len) {
    // ... (Configuración de pines RS485)
    mod.write(comando, len); // Envía trama de petición Modbus
    // ... (Lectura de respuesta)
    if (index == 7 && response[0] == 0x01 && response[1] == 0x03 &&
response[2] == 0x02) {
        uint16_t dato = (response[3] << 8) | response[4]; // Extrae el
valor
        return dato;
    }
    return 0;
}

```

Nota. Elaboración propia.

4.2.3 Procesamiento y transmisión MQTT.

En el bucle principal, el firmware realiza un muestreo de todos los sensores cada 5 segundos y acumula los resultados. Cada 60 segundos, el sistema calcula el promedio de las muestras recolectadas y procede a transmitirlos. La función *publishSensorData()* en *network.cpp* se encarga de empaquetar estos promedios. Utilizando la librería *ArduinoJson*, el sistema construye un objeto JSON estructurado con las variables calculadas y luego serializa el documento en una cadena de caracteres. Finalmente, este payload se publica mediante el cliente MQTT al tópico correspondiente al dispositivo.

Figura 14.

Función para el empaquetamiento y envío de telemetría por MQTT.

```
void publishSensorData(int humidity, int temp, uint16_t &n, uint16_t
&p, uint16_t &k) {
    JsonDocument doc;
    doc["humidity"] = humidity;
    doc["temperature"] = temp;
    doc["nitrogeno"] = n;
    doc["fosforo"] = p;
    doc["potasio"] = k;

    char payload[256];
    serializeJson(doc, payload); // Empaquetado JSON
    mqtt.publish(topicData.c_str(), payload); // Envío MQTT
}
```

Nota. Elaboración propia.

5.3 Identificación única y sistema multinodo

Cada módulo ESP32 utiliza su dirección MAC como identificador único del dispositivo. Este identificador es globalmente único por diseño de hardware y está disponible en tiempo de ejecución a través de *WiFi.macAddress()* una vez que la conexión de red está activa. El **device_id** cumple una triple función en el sistema: (1) constituye parte del *Client ID* del cliente MQTT, garantizando que múltiples módulos puedan conectarse simultáneamente al broker sin conflictos de identificación; (2) es el sufijo del tópico de datos, permitiendo al servidor identificar el origen de cada mensaje sin incluir el ID en el payload; y (3) es el campo de referencia en los documentos de MongoDB que vincula cada lectura y evento de riego con su dispositivo.

Al establecer la conexión MQTT, cada dispositivo publica automáticamente un mensaje de registro en el tópico *device/register* indicando su ID, tipo de planta y estado de conexión.

Para dotar al sistema de escalabilidad y permitir su funcionamiento multinodo —es decir, la capacidad de monitorear múltiples plantas simultáneamente mediante distintos módulos ESP32 de forma independiente— la comunicación se diseñó utilizando tópicos dinámicos. En el firmware, cada módulo construye sus propios canales de publicación y suscripción concatenando su identificador único (deviceId, basado en su dirección MAC) a las rutas base, generando tópicos específicos como *sensor/data/{deviceId}* y *control/led/{deviceId}*.

Por su parte, el servidor backend aprovecha el modelo de suscripción del protocolo MQTT utilizando el comodín # (por ejemplo, suscribiéndose a *sensor/data/#* y *device/status/#*). Esto permite que un único servidor centralizado reciba toda la telemetría y los eventos de conexión de cualquier número de nodos activos en la red. Al recibir un mensaje, el servidor extrae el deviceId directamente de la ruta del tópico, lo que le permite diferenciar de qué módulo provienen los datos, vincularlos con la planta correspondiente en la base de datos y enrutarlos a la interfaz del usuario sin que los datos de múltiples plantas interfieran entre sí

5.4 Servidor Backend e integración con el broker MQTT

El servidor backend, desarrollado en NodeJS y utilizando el framework Express, funciona como un cliente MQTT de alto nivel. Este servidor está suscrito a los tópicos de control y telemetría (`sensor/data/{device_id}`, `device/register` y `control/led/{device_id}`), actuando como puente entre el hardware y la interfaz de usuario. El ESP32 publica sus datos de telemetría con una frecuencia determinada en el tópico de datos. De manera inversa, el dispositivo está suscrito a su tópico de control específico (`control/led/{deviceId}`), lo que le permite recibir comandos desde el servidor para accionar el actuador, encendiéndolo o apagándolo con una latencia mínima.

La lógica de procesamiento de mensajes MQTT se implementa en el manejador `mqttClient.on('message', ...)`, que recibe todos los mensajes de los tópicos suscritos y los enruta según el tópico:

- **device/register:** Al recibir un mensaje en este tópico, el servidor registra o actualiza la información del módulo ESP32 (incluyendo su identificador, estado y tipo de planta asignada) en la base de datos. Tras el registro, el servidor emite inmediatamente un evento tipo `device_update` mediante WebSocket para actualizar la lista de módulos disponibles en el dashboard de los usuarios.
- **sensor/data/#:** Este tópico capta la telemetría continua enviada por todos los módulos activos en la red. Al recibir un mensaje, el servidor extrae el `device_id` directamente de la ruta del tópico y procede a evaluar el estado de salud de la planta cruzando las lecturas físicas con los rangos ideales de su categoría. Luego, almacena un nuevo documento de tipo `SensorReading` en MongoDB con el registro histórico de la lectura y transmite los valores procesados en tiempo real hacia los navegadores conectados.
- **device/status/#:** Es el tópico encargado de monitorear la conectividad de los módulos a través de mensajes de estado o alertas de desconexión abrupta emitidas por el broker. Si el payload indica que un dispositivo ha pasado a estado "offline", el servidor actualiza la base de datos y detiene cualquier evento de riego que estuviera en curso para ese nodo antes de notificar a la interfaz web sobre la desconexión.

En sentido contrario, cuando el servidor WebSocket recibe un mensaje desde el navegador, lo parsea como JSON y publica el comando en el tópico MQTT `control/led/{device_id}`. Cuando el sistema o el usuario activan o detienen la bomba, el servidor envía las instrucciones (`LED_ON` o `LED_OFF`) a este tópico. De manera complementaria, el servidor registra estos comandos en MongoDB creando documentos `WateringEvent` para documentar quién inició el riego y calcular su duración final.

Figura 15.

Bloque de código para manejo de comandos de bomba en el servidor

```
ws.on('message', async (message) => {
  const data = JSON.parse(message.toString());
  const deviceId = data.device_id;
  const command = data.command;

  if (cmd === 'LED_ON') {
```

```

    mqttClient.publish(`control/led/${deviceID}`, 'LED_ON');

    broadcastToClients({ type: 'watering_started', device_id:
deviceID, reason: 'manual' });

    } else if (cmd === 'LED_OFF') {
    mqttClient.publish(`control/led/${deviceID}`, 'LED_OFF');
    // Calcula la duración y actualiza el último evento de riego

    broadcastToClients({ type: 'watering_stopped', device_id:
deviceID, reason: 'manual' });
    {
    });
    });

```

Nota. Elaboración propia.

El servidor WebSocket, instanciado sobre el servidor HTTP de Express, mantiene conexiones persistentes con todos los navegadores que tienen el dashboard abierto. Al recibir un mensaje MQTT (datos de sensor o actualización de dispositivos), el servidor invoca ***broadcastToClients()*** que itera sobre todos los clientes conectados y les envía los mensajes de actualización serializados como JSON.

Figura 15.

Función de conexión websocket con clientes web.

```

function broadcastToClients(data) {
  wss.clients.forEach(client => {
    if (client.readyState === WebSocket.OPEN)
      client.send(JSON.stringify(data));
  });
}

```

Nota. Elaboración propia.

5.5 Base de datos

Para la persistencia de datos, se configuró una conexión con una base de datos no relacional MongoDB utilizando la librería Mongoose, lo que permite definir esquemas tipados para estructurar la información del sistema. La arquitectura de datos se fundamenta en seis modelos principales, cada uno con una funcionalidad específica dentro de la plataforma:

- **User:** Gestiona las cuentas de los usuarios registrados en el sistema. Almacena el nombre de usuario, el correo electrónico y la contraseña.
- **PlantType:** Actúa como el catálogo de referencia del sistema. Define los diferentes tipos de plantas y sus rangos ideales de cultivo, incluyendo parámetros mínimos y máximos para la humedad del suelo, la temperatura ambiente y los macronutrientes NPK (nitrógeno, fósforo y potasio).
- **GardenPlant:** Representa las instancias individuales de plantas añadidas al jardín virtual de un usuario. Este esquema vincula de manera relacional al usuario

- (user_id) con el tipo de planta (plant_type_id). Además, permite asignar de forma opcional un identificador de módulo físico (device_id) a esa planta específica.
- **Device:** Registra los módulos ESP32 emparejados con la plataforma. Utiliza la dirección MAC del microcontrolador como identificador único (device_id) y mantiene el registro del usuario propietario, la planta monitoreada, su estado de conexión actual (online, offline o pending) y su última actividad en la red.
 - **SensorReading:** Es el modelo encargado de almacenar el historial continuo de telemetría. Cada documento representa una lectura puntual recibida vía MQTT, guardando los valores de humedad, temperatura y NPK, así como un estado de salud de la planta calculado en el momento de la lectura (saludable, en riesgo o desconocido) y la lista de alertas generadas.
 - **WateringEvent:** Lleva el registro de las actuaciones de la bomba de riego. Documenta cada vez que se activa el relé, especificando si el riego fue disparado de forma manual desde el dashboard o de manera automática por el servidor al detectar un nivel bajo de humedad. También captura el tiempo total de duración del riego en segundos.

5.6 Sistema de autenticación

Para garantizar la seguridad de la información y restringir el acceso a las funciones de monitoreo y control, el sistema implementa un mecanismo de autenticación basado en JSON Web Tokens (JWT). La arquitectura de este componente se divide en la gestión de credenciales en la base de datos, la emisión de tokens en el servidor y su manejo en la interfaz de usuario.

- **Seguridad de credenciales y base de datos:** El modelo de usuarios (User.js) de la base de datos, construido sobre Mongoose, garantiza que las credenciales se manejen de forma segura, incluyendo un nombre de usuario, dirección de correo electrónico y contraseña. El sistema utiliza la biblioteca bcryptjs para generar y aplicar un hash criptográfico de manera automática antes de guardar cualquier registro en la base de datos de MongoDB para encriptar las contraseñas.
- **Emisión y validación de tokens:** El servidor backend expone las rutas RESTful para el registro (/api/auth/register) y el inicio de sesión (/api/auth/login). Al autenticar correctamente al usuario, el servidor emite un token JWT firmado mediante una clave secreta (JWT_SECRET).
- **Integración en la interfaz de usuario (Frontend):** En la capa de aplicación web, los formularios de acceso son gestionados por un script dedicado (auth.js) que se comunica asíncronamente con la API. Tras un inicio de sesión o registro exitoso, el cliente almacena el token JWT de forma persistente en el almacenamiento local del navegador bajo la clave sp_token.

5.7 Diseño del circuito

La Figura 16 muestra el esquemático del circuito diseñado para el sistema, el cual está

divido en 4 bloques: alimentación, sensado, procesamiento/control y actuador. El circuito es alimentado por una fuente conectada a la red eléctrica que entrega 12V. Este voltaje fue regulado a 5V para alimentar el microcontrolador, el relé y el módulo RS485 por medio de un regulador de voltaje LM338.

Tanto el sensor NPK como el sensor de temperatura/humedad poseen una salida de 4 líneas, las cuales entran al circuito por medio de borneras.

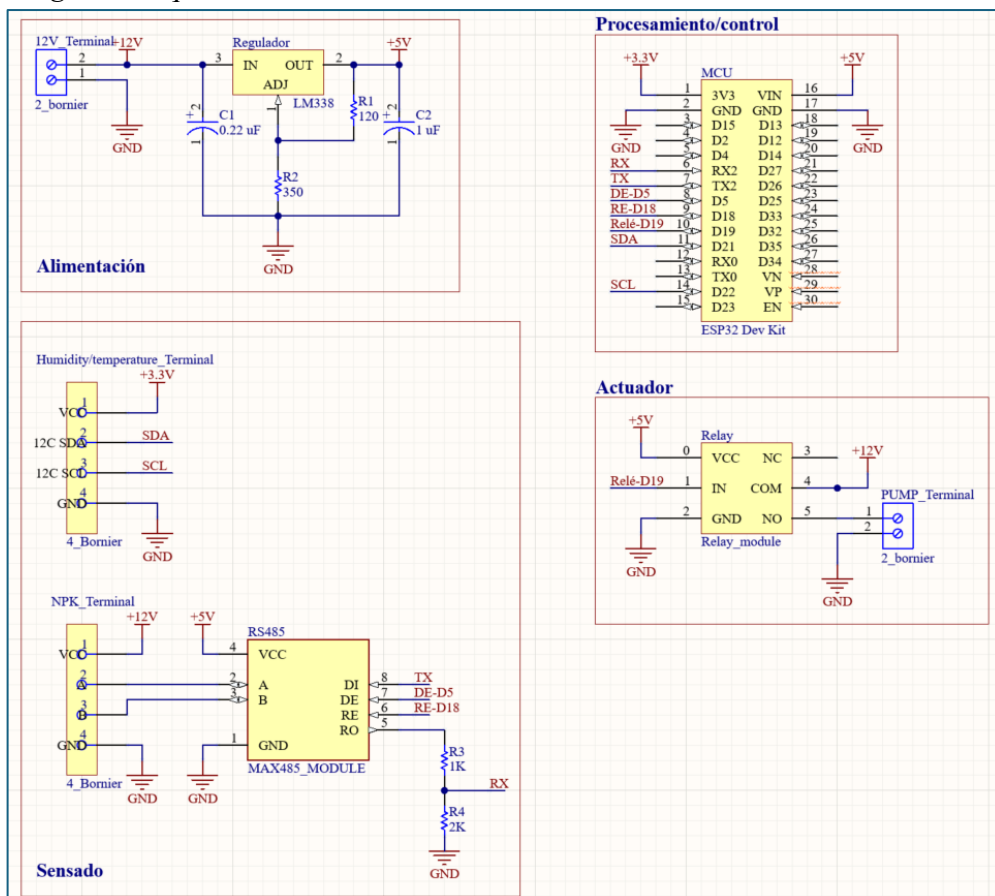
El módulo RS485 se introduce al circuito para poder comunicar el sensor NPK con el ESP32. Este módulo se encarga de traducir la comunicación del sensor NPK, el cual usa Modbus RTU por medio de señales diferenciales entre sus pines A y B, con la comunicación UART del ESP32. Los pines DE (Data Enable) y RE (Receive Enable) del lado del ESP se usan para habilitar la comunicación Half-Duplex.

Dado que el módulo RS485 es alimentado con 5V, sus salidas llegan a alcanzar los 5V. Para no quemar los pines GPIO del ESP32, los cuales están limitados a un voltaje de 3.3V, se usó un divisor de voltaje en la salida del RS485 para asegurar que la entrada al ESP32 no supere su límite de voltaje.

El módulo relé fue necesario para proteger al microcontrolador y aislar la etapa de control de la etapa de potencia de la bomba, dado que esta es alimentada con 12V, por lo que no puede ser controlada directamente con la señal de salida del ESP32.

Figura 16.

Diagrama esquemático del circuito.



Nota. Elaboración propia.

La Tabla 2, por otro lado, muestra todas las conexiones con sus respectivas funciones en el circuito.

Tabla 2.

Tabla de conexiones del circuito.

Componente	Origen	Destino	Descripción
RS485	RE	GPIO18	Habilitación de recepción
	DE	GPIO5	Habilitación de transmisión
	RO	GPIO16 (RX2)	Transmisión de datos del sensor
	DI	GPIO17 (TX2)	Transmisión de comandos RS485
	VCC	5V	Alimentación
Sensor hum/temp	SDA	GPIO21	Línea de datos I2C
	SCL	GPIO22	Línea de sincronización I2C
	VCC	3.3V	Alimentación
Sensor NPK	A	RS485-A	Línea no inversora del bus
	B	RS485-B	Línea inversora del bus
	VCC	12V	Alimentación
Módulo Relay	IN	GPIO19	Señal de control
	VCC	5V	Alimentación
	COM	12V	Salida de voltaje
	NO	(+) de la bomba	Control de la bomba

Nota. Elaboración propia.

5.8 Interfaz de usuario

El objetivo principal de la interfaz es crear un ambiente atractivo para el uso del usuario, permitiéndole registrar sus plantas, visualizarlas en un jardín virtual y monitorear variables importantes del suelo. El sitio web fue desarrollado usando HTML, CSS y JavaScript, y se guarda en el servidor para la actualización del tiempo real de los datos. Se reutilizó una plantilla generada con IA

Log-in y Registro.

El sitio requiere inicio de sesión porque cada jardín virtual, sus plantas y sus lecturas de sensores son datos privados asociados a una cuenta de usuario, por lo que el sistema necesita identificar quién está consultando o modificando esa información antes de mostrar nada. La página presenta dos formularios en pestañas (iniciar sesión / crear cuenta) gestionados por auth.js, que envían los datos mediante fetch a /api/auth/login o /api/auth/register. La contraseña nunca se guarda ni viaja en texto plano: el servidor la

recibe, la cifra mediante una función de hash unidireccional y almacena únicamente ese hash en la base de datos, de modo que ni siquiera el propio sistema puede leer la contraseña original, solo comparar hashes en cada intento de acceso. Si las credenciales son correctas, el servidor responde con un token JWT que el navegador guarda en localStorage bajo la clave sp_token; ese token es lo que se reenvía en el encabezado Authorization de cada petición protegida durante la sesión, y es también lo que verifica el servidor para identificar al usuario y filtrar solo sus datos.

Figura 17.
Pantallas de la interfaz web.

	
Log-in y Sign-up	Home page
	
Add plant	Virtual garden
	
Monitoreo	Cuenta del usuario

Home.

Funciona como la página de bienvenida y carta de presentación del proyecto: resume visualmente el flujo planta → sensores → ESP32 → WiFi → servidor → plataforma web y describe los componentes de hardware (sensor de humedad, sensor NPK, bomba de riego, ESP32). Es una página estática, sin datos dinámicos del usuario, su propósito es orientar a cualquier visitante sobre qué hace el sistema antes de entrar a la cuenta.

Add Plant.

Es el catálogo del sistema: presenta las especies disponibles agrupadas por categoría (follaje, florales, desérticas, vegetales y frutales), cada una representada con un modelo 3D interactivo y navegable en carrusel. Al hacer clic sobre la tarjeta de una especie, se envía un POST a `/api/garden` que crea una nueva instancia de esa planta asociada a la cuenta del usuario; como cada clic genera una instancia nueva, es posible añadir la misma especie tantas veces como se desee, y cada copia se monitorea de forma independiente.

Los modelos 3D que fueron usados para el catálogo de plantas fueron sacados de un sitio web llamado Sketchfab.com que permite la descarga gratuita de las piezas. Y el código para generar los carruseles en ese estilo no fueron diseñados desde cero, sino que el código base fue sacado de un sitio web llamado Codepen.io que permite la descarga de ejemplos de código gratuito para sitios web.

Virtual Garden.

Es la página que centraliza el jardín del usuario: al cargar hace un GET a `/api/garden` y dibuja dinámicamente una tarjeta 3D por cada planta guardada. Al seleccionar una planta se abre un panel con las acciones disponibles: abrir datos de monitoreo, designar el módulo físico vinculado o eliminar la planta. Esta página es, en la práctica, el punto de unión entre el resto del sitio: como ninguna página guarda las plantas en el navegador, sino que todas consultan el mismo endpoint `/api/garden` en el servidor con el token del usuario, cualquier cambio hecho en Add Plant o en Virtual Garden (agregar, eliminar, asignar un sensor) se refleja de inmediato tanto en Cuenta como en Monitor sin necesidad de sincronización manual entre páginas.

6. Implementación física del Sistema

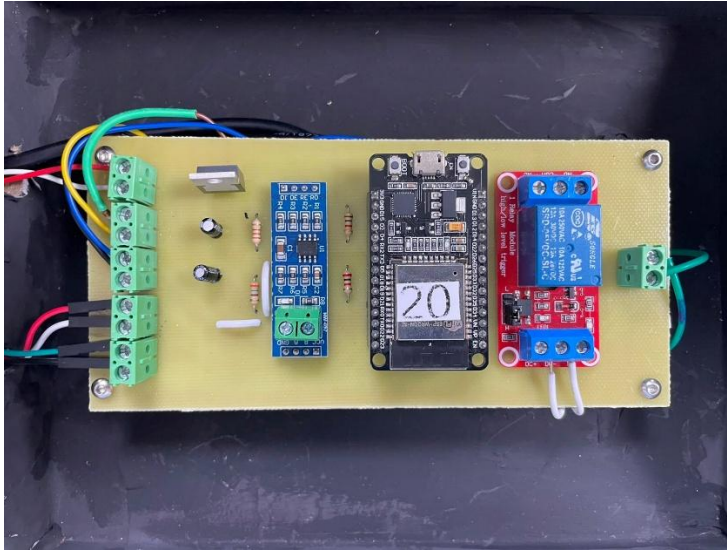
Una vez validado el diseño esquemático descrito en la sección 5.7, se procedió al ensamblaje físico del circuito y su integración con los sensores y el actuador. Esta etapa permitió confirmar que las conexiones, la regulación de voltaje y la comunicación entre módulos funcionan correctamente fuera del entorno de simulación.

La Figura 18 muestra el circuito principal impreso y ensamblado sobre una placa de baquelita. En ella se identifican los bloques descritos previamente: la bornera de alimentación y las conexiones de los sensores (izquierda), el regulador de voltaje LM338 con sus capacitores de filtrado, el módulo RS485/TTL (MAX485) para la comunicación con el sensor NPK, el microcontrolador ESP32 como unidad central de procesamiento, y el módulo relé de un canal que controla la activación de la bomba de riego.

Por su parte, la Figura 19 muestra el nodo de sensado de humedad, temperatura y nutrientes, correspondiente al sensor capacitivo y el sensor NPK descritos, ya utilizados en pruebas directas sobre sustrato. El empaque fue diseñado e impreso en 3D para integrar los dos sensores.

Figura 18.

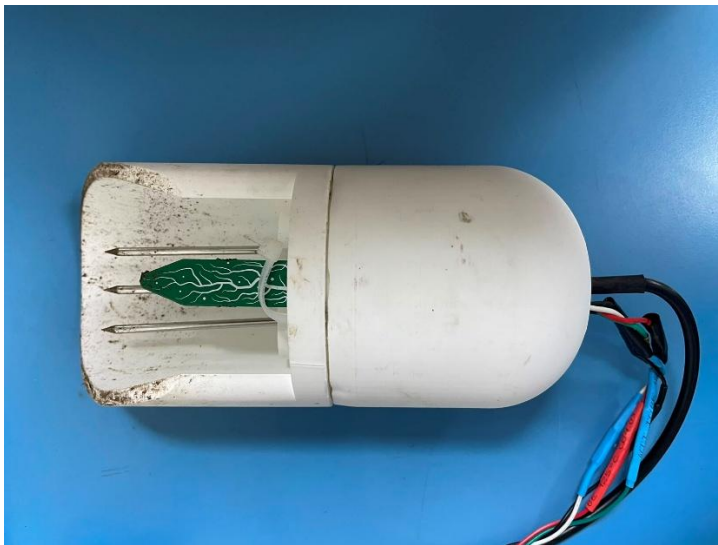
Circuito principal ensamblado.



Nota. Elaboración propia.

Figura 19.

Nodo de sensado de humedad y temperatura.



Nota. Elaboración propia.

Finalmente, la Figura 20 presenta el montaje integrado del sistema completo, donde se observa la carcasa que aloja el circuito principal, conectada mediante mangueras al depósito de agua y a la bomba peristáltica, junto con la fuente de alimentación externa y los sensores ya cableados al conjunto. Esta configuración corresponde al prototipo utilizado para las primeras pruebas funcionales de riego automatizado.

Figura 20.

Montaje integrado del sistema (circuito, bomba y depósito de agua).



Nota. Elaboración propia.

El funcionamiento del sistema en condiciones reales —incluyendo la lectura de sensores y la activación de la bomba— fue registrado en un video demostrativo disponible en el siguiente enlace: [video demo smart-plant](#).

Estas pruebas confirman la viabilidad física de la arquitectura propuesta, validando los objetivos y sentando las bases para la integración del sistema.

7. Conclusiones

El desarrollo del proyecto permitió completar un sistema IoT funcional de monitoreo ambiental y riego automático de plantas, validando de extremo a extremo la comunicación entre el hardware de adquisición, el broker MQTT, el servidor backend y la interfaz web. Se logró implementar y verificar el funcionamiento conjunto de los módulos ESP32 con los sensores de humedad, temperatura y nutrientes (NPK), la transmisión cifrada de telemetría vía TLS/SSL hacia el broker EMQX Cloud, el procesamiento y almacenamiento de las lecturas en MongoDB, y su visualización en tiempo real a través de un dashboard web con autenticación de usuarios y comunicación WebSocket.

La arquitectura modular y multinodo, basada en la identificación de dispositivos por dirección MAC y en el uso de tópicos MQTT dinámicos, cumplió su objetivo de permitir que un único servidor gestione simultáneamente múltiples nodos de monitoreo sin interferencia entre sus datos. Asimismo, la base de datos de umbrales fundamentada científicamente para veinticinco especies distribuidas en cinco categorías ecológicas permitió automatizar la evaluación del estado de salud de cada planta y sustentar las alertas generadas por el sistema. El circuito físico, que integra el regulador de voltaje LM338, el módulo RS-485/TTL y el relé de estado sólido, demostró ser capaz de controlar de forma segura la activación de la bomba peristáltica en respuesta a las condiciones de humedad detectadas.

En conjunto, estos resultados confirman que Smart-Plant cumple con el objetivo

general propuesto: un sistema embebido IoT modular, gestionado a través de internet mediante una interfaz de usuario interactiva, capaz de monitorear variables ambientales y de nutrientes y de ejecutar el riego automatizado en entornos domésticos y de oficina. El proyecto podría considerarse como una alternativa modular, escalable y accesible frente a las soluciones existentes, generalmente orientadas a un único nodo o a un control de riego basado exclusivamente en humedad, sin capacidad de gestión multinodo ni diagnóstico ambiental integral.

8. Limitaciones y líneas futuras

Si bien el sistema demostró ser técnicamente viable, el desarrollo del proyecto permitió identificar un conjunto de limitaciones que constituyen líneas de trabajo futuro para fortalecer la viabilidad y utilidad del proyecto.

- **Precisión y confiabilidad del sensor de nutrientes (NPK).** El sensor utilizado no ha sido validado contra un valor de referencia independiente. Se recomienda, como trabajo futuro, contrastar las lecturas del sensor contra un análisis de laboratorio o soluciones de calibración certificadas para determinar su error y aplicar una curva de corrección.
- **Calibración del sensor de humedad/temperatura.** El sensor capacitivo fue calibrado mediante un método de dos puntos, registrando el valor de capacitancia en tierra completamente seca y completamente húmeda para un sustrato específico. Esta calibración empírica no ha sido validada frente a un método de referencia ni contrastada en distintos tipos de sustrato, cuyas propiedades dieléctricas pueden variar y afectar la exactitud del porcentaje de humedad reportado fuera del sustrato usado para calibrar.
- **Dependencia de conexión a internet para el riego automático.** El sistema depende de una conexión WiFi estable para transmitir la telemetría al broker EMQX Cloud y recibir los comandos de actuación de la bomba. No existe un modo de operación local que permita ejecutar el riego automático a partir de umbrales cuando la conexión a internet no está disponible. Como línea futura se propone implementar un modo de local, evaluando la humedad y activando el riego directamente en el ESP32 cuando se detecte la pérdida de conectividad.
- **Autonomía energética del módulo.** El diseño actual, basado en el regulador LM338, opera únicamente conectado a una fuente de alimentación externa, sin batería de respaldo. Esto limita el despliegue del sistema a ubicaciones con acceso permanente a un tomacorriente y expone al sistema a interrupciones en caso de cortes eléctricos. Se recomienda evaluar la incorporación de una batería recargable con circuito de carga, acompañada de un análisis de consumo del ESP32 y los sensores para determinar la autonomía.
- **Configuración de red WiFi estática en el firmware.** Las credenciales de red están definidas como constantes en el código fuente del firmware, lo que obliga a recompilar y re-flashear el ESP32 cada vez que el dispositivo deba conectarse a una red distinta. Esto limita la escalabilidad y la facilidad de instalación del

sistema para un usuario final sin conocimientos técnicos. Como mejora futura se sugiere implementar un mecanismo de aprovisionamiento de credenciales en tiempo de ejecución o configuración vía Bluetooth/BLE desde una aplicación móvil.

Referencias

- Acosta Gallo, E. D., & Cuaical Angulo, C. R. (2021). Desarrollo de un sistema automático de irrigación para huertos caseros por medio de IoT con software y hardware libre [Trabajo de titulación, Universidad Politécnica Salesiana]. <https://dspace.ups.edu.ec/handle/123456789/20382>
- Adafruit Industries. (2024, 8 de marzo). Adafruit STEMMA Soil Sensor: I2C capacitive moisture sensor. Adafruit Learning System.
- American Orchid Society. (2023). Orchid care: Fertilizing. <https://www.aos.org/orchids/orchid-care/fertilizing.aspx>
- Angulo Martínez, W. A. (2025). Desarrollo e implementación de un sistema de riego inteligente basado en internet de las cosas para cultivos protegidos en Aguascalientes, México [Tesis de maestría, Universidad Autónoma de Aguascalientes]. 483563.pdf
- Brady, N. C., & Weil, R. R. (2016). The nature and properties of soils (15th ed.). Pearson.
- Epstein, E., & Bloom, A. J. (2005). Mineral nutrition of plants: Principles and perspectives (2nd ed.). Sinauer Associates.
- Gómez Zea, J. M., Falcón Pérez, F., Jesús Magaña, J. Á., Carrera Belueta, C. M., González de la Cruz, R., Torres Hernández, J. J., & de la Cruz Alvarez, J. (2025). Sistema de riego por aspersión IoT para producción controlada de forraje verde hidropónico en traspatio, como alternativa para la autosuficiencia alimentaria. *Innovación y Desarrollo Tecnológico Revista Digital*, 17(3), 1397. https://iydt.wordpress.com/wp-content/uploads/2025/08/3_45_sistema-de-riego-por-aspersion-iot-para-produccion-controlada-de-forraje-verde-hidroponico-en-traspatio-como-alternativa-para-la-autosuficiencia-alimentaria.pdf
- Gondchawar, N., & Kawitkar, R. S. (2016). IoT based smart agriculture. *International Journal of Advanced Research in Computer and Communication Engineering*, 5(6), 838–842. <https://doi.org/10.17148/IJARCCCE.2016.56213>
- Hasib, A., & Ahsanul Sarkar Akib, A. S. M. (2026). An IoT-based smart plant monitoring and irrigation system with real-time environmental sensing, automated alerts, and cloud analytics. *arXiv*. <https://arxiv.org/abs/2601.15830>
- Hernández González, A. G., Hernández Sánchez, D. L., García Vázquez, O. O., & Fernández Jiménez, J. L. (2025). Sistema de riego automatizado con Raspberry Pi, ESP32 y comunicación vía MQTT y HTTP. *Revista Ingeniantes*, 1(2), 61.
- Introduction of message queue telemetry transport protocol (MQTT). (2020, July 23). *GeeksforGeeks*. <https://www.geeksforgeeks.org/computer-networks/introduction-of-message-queue-telemetry-transport-protocol-mqtt/>
- Jones, H. G. (2014). *Plants and microclimate: A quantitative approach to environmental plant physiology* (3rd ed.). Cambridge University Press.
- lily-osp. (2025). smart-farming-esp32. *GitHub*. <https://github.com/lily-osp/smart-farming->

- Marschner, P. (Ed.). (2012). Marschner's mineral nutrition of higher plants (3rd ed.). Academic Press.
- Oppong, R. A. (2025). Integration of IoT-based, sprinklers, embedded systems, data and cloud computing for smart irrigation management. *World Journal of Advanced Research and Reviews*, 25(3), 126–151. <https://doi.org/10.30574/wjarr.2025.25.3.0695>
- Perea Alvarado, F. C., & Bustamante Briones, G. A. (2024). Prototipo de sistema de riego automatizado con control de humedad por histéresis y monitoreo IoT para la reducción del recurso hídrico [Trabajo de titulación, Universidad Politécnica Salesiana]. <https://dspace.ups.edu.ec/handle/123456789/30472>
- Prokosch, T. (2025, 17 de junio). MQTT protocol explained: The complete guide. Cedalo. <https://www.cedalo.com/blog/complete-mqtt-protocol-guide>
- ¿Qué es MongoDB? (s.f.). MongoDB. <https://www.mongodb.com/es/docs/manual/>
- Richard-Roman. (2026). RiegoJardin-ESP32. GitHub. <https://github.com/Richard-Roman/RiegoJardin-ESP32>
- Ros López, D. (2021). Diseño y desarrollo de una aplicación web de monitorización de dispositivos Bluetooth Low Energy [Trabajo Fin de Grado, Universidad Politécnica de Cartagena]. Nombre del Repositorio. URL
- Ruiz, F. Y. (2025). Smart irrigation system using IoT and LSTM for optimal water management [Tesis de maestría, University of Texas at Arlington]. MavMatrix. https://mavmatrix.uta.edu/electricaleng_theses/399
- Sonneveld, C., & Voogt, W. (2009). Plant nutrition of greenhouse crops. Springer. <https://doi.org/10.1007/978-90-481-2532-6>
- Strik, B. C. (2016). Nutrient management of organic blueberry. Oregon State University Extension Service.
- Taiz, L., Zeiger, E., Møller, I. M., & Murphy, A. (2015). Plant physiology and development (6th ed.). Sinauer Associates.
- Tovar Soto, J. P., Solórzano Suárez, J. S., Badillo Rodríguez, A., & Rodríguez Cainaba, G. O. (2019). Internet de las cosas aplicado a la agricultura: estado actual. *Lámpsakos*, (22), 86-105. <https://doi.org/10.21501/21454086.3253>
- VanDerZanden, A. M. (2008/2024). Environmental factors affecting plant growth. Oregon State University Extension Service.
- WebSocket and its difference from HTTP. (2019, 4 de diciembre). GeeksforGeeks. <https://www.geeksforgeeks.org/web-tech/what-is-web-socket-and-how-it-is-different-from-the-http/>
- Zamora Re, M. I., Peplin, T. M., & Tomasek, A. (2026). Soil moisture monitoring to support irrigation scheduling (EM 9868). Oregon State University Extension Service.